

Сравнение моделей конкурентности: Python vs Go

Python vs Go: философия

- **Парадигма:** мультипарадигменный язык vs конкурентно-ориентированный язык
- **Типизация:** динамическая vs статическая
- **Выполнение:** интерпретируемый vs компилируемый
- **Память:** GC с GIL vs GC без GIL
- **Конкурентность:** пользовательская (asyncio) vs встроенная в рантайм

Генераторы vs Горутины

Генераторы в Python:

- работают внутри одного потока
- переключаются через yield
- переключение = переход внутри интерпретатора

```
def gen():  
    for i in range(10):  
        yield i
```

Горутины в Go:

- легковесная версия потоков
- управляются планировщиком
- переключение = блокировка через каналы или синхронизация

```
func goroutine() {  
    for i := 0; i < 10; i++ {  
        fmt.Println(i)  
    }  
}
```

План

- Стоимость одной итерации
- Масштабируемость (глубина цепочки)
- Потребление памяти
- Throughput и Latency под нагрузкой (массовый запуск задач)

Стоимость переключения контекста

Особенности:

- Python переключается между **фреймами**, легкая операция
- Go переключается между **горутинами** через **channel**, требует синхронизации

Метрики:

- среднее время одной итерации (ns)

Скорость переключения контекста

```
from time import perf_counter

N = 10**8

def gen():
    while True:
        yield

g = gen()

# прогон для прогрева
for _ in range(1000):
    next(g)

# замер времени выполнения N итераций
start = perf_counter()
for _ in range(N):
    next(g)
gen_time = perf_counter() - start

# подсчет среднего
print(f"Среднее время итерации (в нс): {gen_time / N * 1_000_000_000}")
```

```
package main

import (
    "fmt"
    "time"
)

func main() {
    const N = 100000000

    ch := make(chan struct{})

    go func() {
        for {
            ch <- struct{}{}
        }
    }()

    // прогон для прогрева
    for i := 0; i < 1000; i++ {
        <-ch
    }

    // замер времени выполнения N итераций
    start := time.Now()
    for i := 0; i < N; i++ {
        <-ch
    }
    elapsed := time.Since(start)

    // # подсчет среднего
    avgNs := float64(elapsed.Nanoseconds()) / N
    fmt.Printf("Среднее время итерации (в нс): %.2f\n", avgNs)
}
```

Скорость переключения контекста

- Среднее время итерации в Python: 34.44 наносекунд
- Среднее время итерации в Go: 96.78 наносекунд

Почему так:

- Python проверяет состояние генератора, переключается на фрейм генератора, возвращается на основной фрейм
- Go блокирует мьютекса канала, основная горутин засыпает, горутин функции просыпается, горутин функции засыпает, основная горутин просыпается
- Операции с каналом в Go дороже прямого переключения в Python

Масштабируемость (глубина цепочки)

Особенности:

- Python использует **yield from**, делегируя выполнение внутри VM
- Go строит **пайплайн** из горутин и каналов, блокируя и синхронизируя

Метрики:

- зависимость времени выполнения от глубины цепочки

Масштабируемость (глубина цепочки)

```
from time import perf_counter

MAX_D = 100
N = 10**6

def base():
    yield 1

def make_chain(d):
    def gen():
        yield 1

    for _ in range(d):
        prev = gen
        def gen(prev=prev):
            yield from prev()
        return gen

depths = []
times = []

for d in range(1, MAX_D + 1):
    print(f"Testing depth {d}...")
    g = make_chain(d)()

    start = perf_counter()
    for _ in range(N):
        try:
            next(g)
        except StopIteration:
            g = make_chain(d)()
    t = perf_counter() - start

    depths.append(d)
    times.append(t)

with open("python_chain.txt", "w") as f:
    for d, t in zip(depths, times):
        f.write(f"{d} {t}\n")
```

```
const MAX_D = 100
const N = 1000000

func stage(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for v := range in {
            out <- v
        }
        close(out)
    }()
    return out
}

func main() {
    file, _ := os.Create("go_chain.txt")
    defer file.Close()

    for d := 1; d <= MAX_D; d++ {
        start := time.Now()

        for i := 0; i < N; i++ {
            var ch <-chan int

            base := make(chan int, 1)
            base <- 1
            close(base)

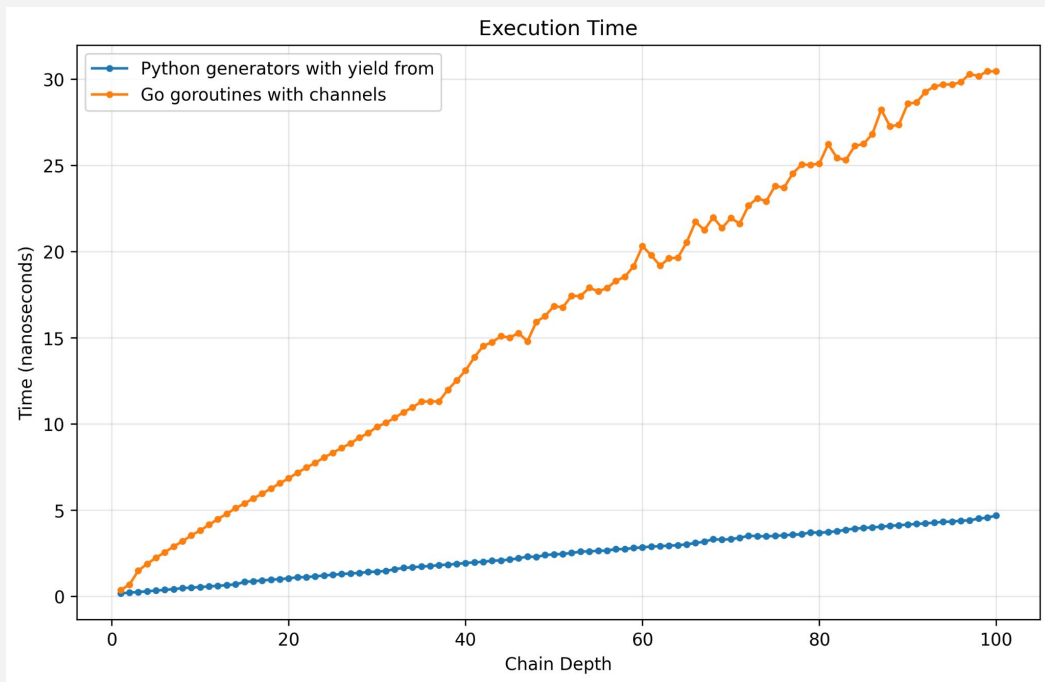
            ch = base

            for j := 0; j < d; j++ {
                ch = stage(ch)
            }

            for range ch {
            }

            elapsed := time.Since(start)
            fmt.Printf("%d %f\n", d, elapsed.Seconds())
            fmt.Fprintf(file, "%d %f\n", d, elapsed.Seconds())
        }
    }
}
```

Масштабируемость (глубина цепочки)



Масштабируемость (глубина цепочки)

Почему так:

- В Python не создает новые сущности на каждом этапе, он просто переходит по указателям внутри VM
- В Go на каждый новый уровень создается новая горутина, новый канал, каждый переход — дорогое взаимодействие с каналом

Потребление памяти

Особенности:

- Python использует один активный фрейм независимо от количества вызовов
- Go требует выделения отдельного стека горутины и структуры канала

Метрики:

- пиковое потребление памяти (MB)

Потребление памяти

```
import tracemalloc

MAX_N = 100000
STEP = 1000

def gen():
    yield 1

sizes = []
memory = []

tracemalloc.start()

for n in range(1, MAX_N + 1, STEP):
    gens = [gen() for _ in range(n)]
    current, peak = tracemalloc.get_traced_memory()

    sizes.append(n)
    memory.append(peak)

    del gens

with open("python_memory.txt", "w") as f:
    for s, m in zip(sizes, memory):
        f.write(f"{s} {m}\n")
```

```
const MAX_N = 100000
const STEP = 1000

func worker() {
    select {}
}

func main() {
    file, _ := os.Create("go_memory.txt")
    defer file.Close()

    var m runtime.MemStats

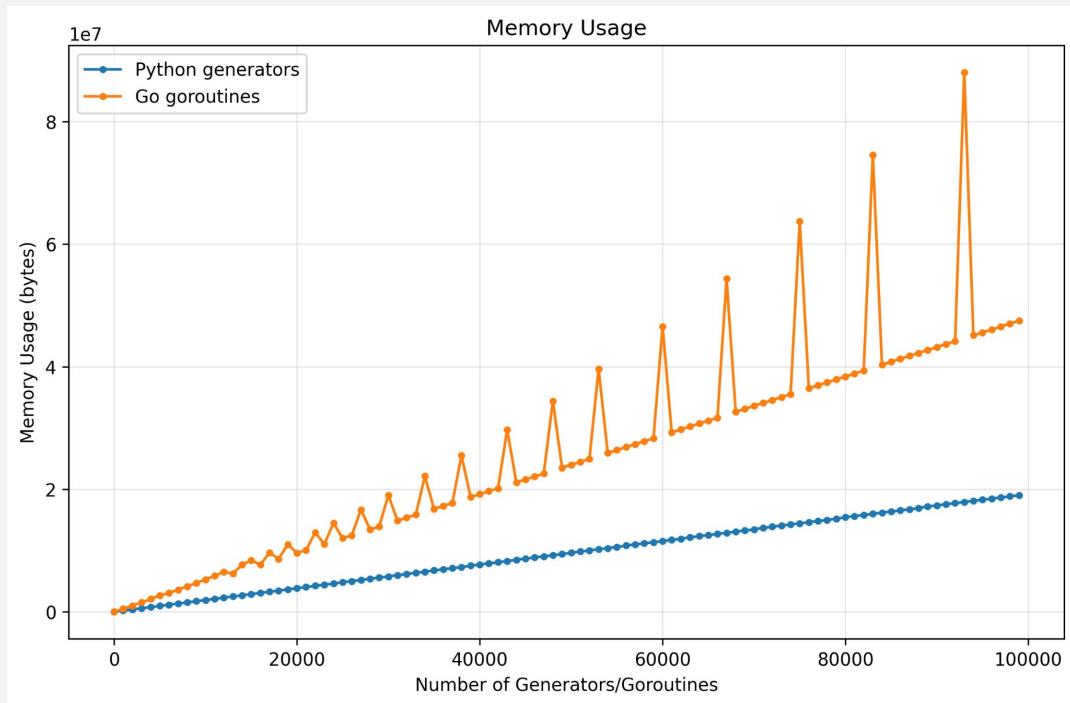
    for n := 1; n <= MAX_N; n += STEP {
        runtime.GC()
        runtime.ReadMemStats(&m)
        before := m.Alloc

        for i := 0; i < n; i++ {
            go worker()
        }

        runtime.ReadMemStats(&m)
        after := m.Alloc

        fmt.Fprintf(file, "%d %d\n", n, after-before)
    }
}
```

Потребление памяти



Потребление памяти

Почему так:

- Python хранит только один объект генератора с минимальным состоянием выполнения внутри одного фрейма
- Go создаёт отдельную горутину с собственным стеком и runtime-структурами для каждой задачи.
- Go также имеет сложный планировщик MPG, который требует хранить горутин

Throughput и Latency под нагрузкой

Особенности:

- В Python `asyncio` выполняет задачи в одном потоке
- В Go горутины распределяются по нескольким потокам

Метрики:

- общее время выполнения N независимых задач

Throughput и Latency под нагрузкой

```
import asyncio
from time import perf_counter

MAX_N = 100000
STEP = 1000

async def worker():
    await asyncio.sleep(0.001)

async def run(n):
    start = perf_counter()
    await asyncio.gather(*[worker() for _ in range(n)])
    total = perf_counter() - start

    throughput = n / total
    latency = total / n * 1e9

    return throughput, latency

async def main():
    with open("python_async.txt", "w") as f:
        for n in range(1, MAX_N + 1, STEP):
            print(f"Running with {n} workers...")
            t, l = await run(n)
            f.write(f"{n} {t} {l}\n")

asyncio.run(main())
```

```
const MAX_N = 100000
const STEP = 1000

func worker(wg *sync.WaitGroup) {
    time.Sleep(time.Millisecond)
    wg.Done()
}

func run(n int) (float64, float64) {
    var wg sync.WaitGroup

    start := time.Now()

    for i := 0; i < n; i++ {
        wg.Add(1)
        go worker(&wg)
    }

    wg.Wait()

    total := time.Since(start).Seconds()

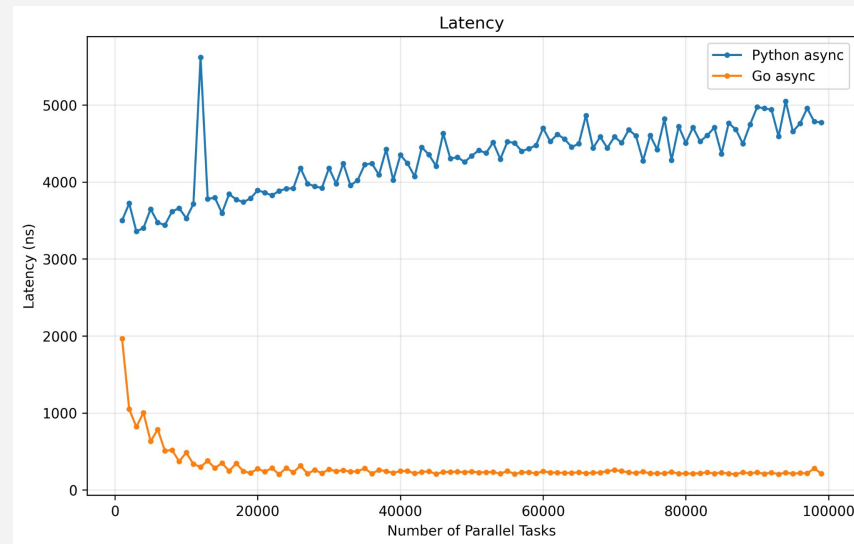
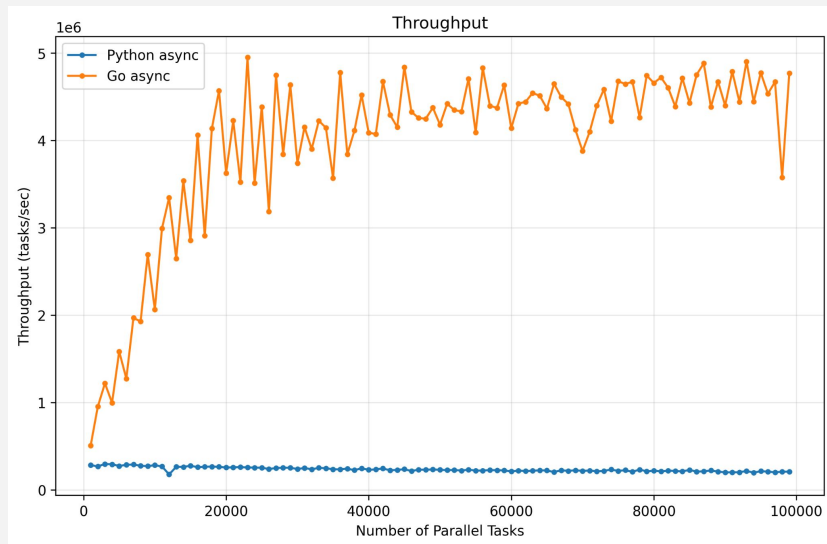
    throughput := float64(n) / total
    latency := total / float64(n) * 1e9

    return throughput, latency
}

func main() {
    file, _ := os.Create("go_async.txt")
    defer file.Close()

    for n := 1; n <= MAX_N; n += STEP {
        fmt.Printf("Running with %d goroutines...\n", n)
        t, l := run(n)
        fmt.Fprintf(file, "%d %f %f\n", n, t, l)
    }
}
```

Throughput и Latency под нагрузкой



Throughput и Latency под нагрузкой

Почему так:

- Python ограничен GIL, поэтому задачи выполняются кооперативно в одном потоке без реального параллелизма
- Go использует многопоточную модель исполнения и может распределять горутины по нескольким ядрам процессора
- Это позволяет Go масштабировать throughput, но увеличивает стоимость планирования и переключения задач

Итоги

	Python	Go
небольшое число задач	просто и дёшево	избыточный runtime
большое число задач	упирается в GIL	масштабируется
однопоточная обработка	эффективен	лишние накладные расходы
параллельная обработка	нет настоящего параллелизма	использует несколько потоков

Вопросы